

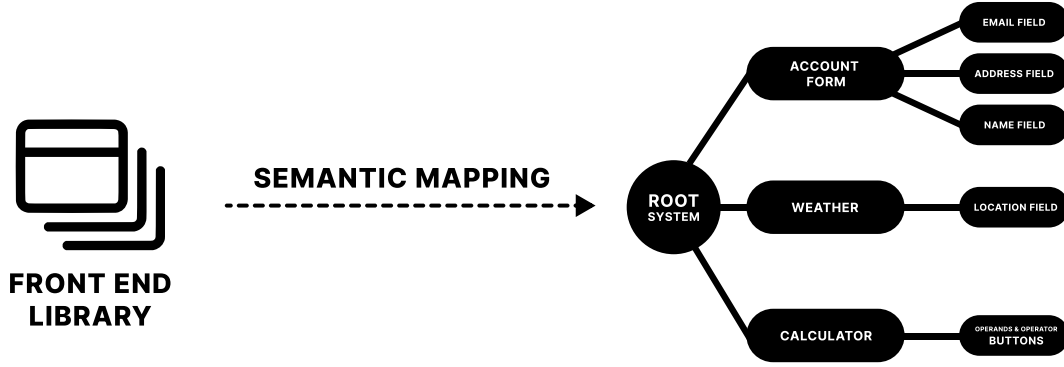


Figure 4: Each application is semantically mapped into the tree structure. Each node holds detailed annotations

an event-driven environment where behaviours and states are modifiable based on user interactions conveyed through natural language prompts.

4.2.1 Annotation Tree & Meta Descriptions

The data structure storing meta descriptions of each UI component in an application is the key principle behind the proposed framework. Our requirement of this textual modelling is the crucial link bridging the gap between contemporary user interfaces and the emergence of highly advanced LLMs. These annotations provide detailed specifications of how each UI component fits conceptually within the structure of the application, what its configurable parameters are, and examples of potential use cases. This information primes applications for communication with the back-end engine’s agents further down the pipeline.

Let us denote our application library system as S , where S contains all elements representing applications and their components:

$$S = \{A, W, C\}$$

where A , W , and C denote the sets for the Account Form, Weather, and Calculator applications from our application library, respectively. Let each nested application set be denoted as:

$$A = \{a_1, a_2, a_3, \dots\} \text{ for Account Form components}$$

$$W = \{w_1, w_2, w_3, \dots\} \text{ for Weather components}$$

$$C = \{c_1, c_2, c_3, \dots\} \text{ for Calculator components}$$

where each contained element represents a unique node modelling a UI component. Each node stores annotations describing the component’s role in the application, along with the descriptors of what parameters it manipulates. See *Listing 1.2*

Listing 2: An example of how an application’s component metadata is stored as a node in the annotation tree data structure

```

a1: {
  "AppName": "Weather",
  "Component Name": "Location Field"
  "Description": "A sub-node of Weather. Represents the point where location
    or query details are input for weather-related inquiries."
  "Parameters": { "City": "What is the location?" }
},
...}
  
```

4.3 Parsing & Mapping User Input

The user provides input describing their target intention through the system’s textual prompt input field or through a speech button which triggers a recording of the user’s spoken prompt. Once received and converted to text through speech recognition models, if speech is used, a prompt is sent to the back end where it is processed by an agent-based prompting mechanism. The processing pipeline carries out appropriate **encoding**, **tokenization**, and finally **mapping via classification**.

Tokenization

1. An input sentence is split into tokens, adhering to an agent’s specific LLM encoding requirements. We will cover BERT and its use of WordPiece tokenization for this example
2. Special tokens like ‘[CLS]’ (at the beginning) and ‘[SEP]’ (at the end) are added
3. The tokenization can be represented as a function that converts a sentence into a sequence of tokens:

$$\text{Tokenize}(\text{"Input Sentence"}) = ["[CLS]", \text{"Token}_1, \text{"Token}_2, \dots, \text{"Token}_N, "[SEP]"]$$

Encoding

1. Each token is then mapped to an ID using BERT’s vocabulary. This can be represented as

$$\text{ID}_i = \text{Vocab}(\text{"Token}_i"), \text{ for } i = 1, 2, \dots, N$$

where ‘Vocab’ is a function that returns the ID of a token in the BERT vocabulary.

2. The total embedding for a token, E_{token} , includes word, segment, and positional embeddings.

4.3.1 Mapping via Classification

To classify user input, we encode the input and node representations:

- For user input $u_i : v_{u_i} = \text{encode}(\text{tokenize}(u_i))$
- For a node n_j representing an application or component: $v_{n_j} = \text{encode}(\text{tokenize}(n_j))$

Let U denote the user’s encoded input:

$$U = \{u_1, u_2, u_3 \dots\}$$

The system then maps user input to system nodes using an interpretation function that performs classification, $f : U \rightarrow S$, where:

$f(U_i) = n_k$, where n_k is chosen such that:

$$n_k = \text{argmax}(\text{cosine_similarity}(\vec{v}_{u_i}, \vec{v}_{n_j}))$$

This process classifies the user’s input to the most relevant application component in the annotation tree. To handle complex inputs that involve specific actions or components, we extend the function f to further map inputs to nested sets (components) within A , W , or C :

$$f(u_i) = a_j \in A \text{ or } f(u_i) = w_j \in W \text{ or } f(u_i) = c_j \in C$$

This implies that the function not only identifies the relevant application but also the specific action or component within that application.

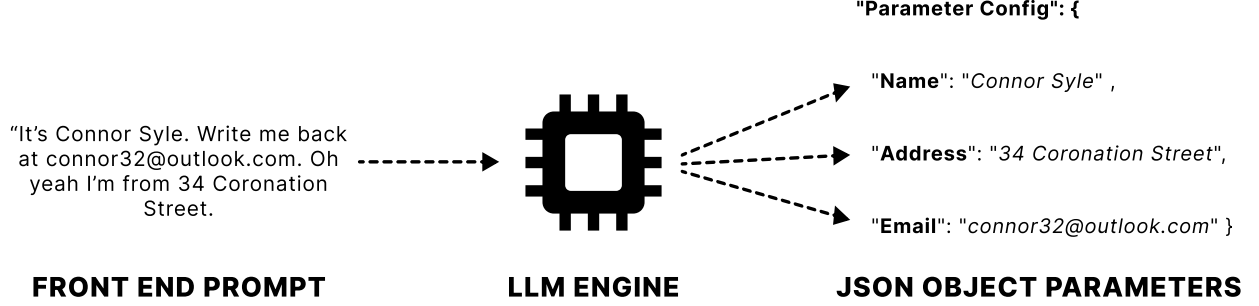


Figure 5: Entity Extraction & Parameterization

4.3.2 Entity Extraction and Parameterization via LLM Inference

At this level, the custom fine-tuned LLM agents with specialized ability such as Google’s T5 model trained on specific tasks from our library in a teacher-student environment, enable accurate parsing of complex user inputs into structured data that the system can act upon. This process is crucial for interpreting the specific details contained within the user’s input and mapping them to actionable UI component parameters in our system.

For inputs that require extracting parameters or entities, we define a transformation $g: U \rightarrow P$, where P is a set of parameters extracted from the inputs. This can be used alongside f to not only map an input to an application component but also extract the necessary information to perform the action:

$$g(u_i) = \{p_1, p_2, \dots\}$$

where u_i is the encoded user input, and $\{p_1, p_2, \dots\}$ are the extracted parameters. These parameters are then used to dynamically adapt the system’s response to the user’s intent. This extraction algorithm functions as a decision-making tool with the use of natural language processing techniques. Its objective is to analyze textual input, categorize it based on predefined criteria, and generate a JSON parameter extraction object to be relayed to the front end and dispatched to the Redux central store.

4.3.3 Action Execution Based on Mapping

Finally, we use the outcomes of f and g to create a final JSON object and execute actions within the system. This involves sending the final JSON object to the front end where the Redux central store invokes specific methods associated with the identified components, passing the extracted parameters and dynamically updating the UI. Let the following represent the introduction of relevant methods:

$$execute(f(u_i), g(u_i))$$

4.3.4 Example:

Assume a user input u_i = “I have a flight to France, what’s it like out there?”. The interpretation function f maps this input to the weather application’s “location” field component $w_{location} \in W$, and g extracts “France” as a parameter. The system then executes the location-based action for “France” based on this mapping and parameter extraction.

The defined detailed approach allows the system to dynamically interpret user inputs by understanding their intent and mapping them to the most relevant application and components in the application library system. It ensures that user inputs are handled efficiently, navigating through the hierarchical structure of the application library based on semantic similarity, thereby improving user experience through accurate identification and interpretation of a user’s needs.

4.4 Operational Workflow

1. **Input Analysis** The algorithm initially analyzes the input to determine its nature and relevance to each UI component child node under the classified application. This categorization is crucial in deciding the subsequent processing path.